

CarLog Pro — état d'avancement actualisé

Mise à jour du 27 septembre 2026 · Cette synthèse actualise les statuts antérieurs du dossier, qui indiquaient notamment que l'interface Web était encore planifiée.

État des phases

Phase	État	Constat actuel
0 · Socle	Largement en place	Architecture, CLAUDE.md, workflow Replit, variables et séparation des bases documentés. Preview et choix final du design restent à valider.
1 · Backend	Largement terminé	Authentification, isolation, RBAC et règles métier implémentés ; tests automatisés passent. Validation manuelle multi-entreprises et feu vert restent ouverts.
2 · Auth Web	Implémentée	Inscription réelle, connexion, session restaurée et gestion du token refusé présentes. Parcours réel dans Preview encore à exécuter.
3 · Véhicules	Fonctions principales faites	CRUD, filtres, archivage, permissions et validation PTAC présents. Vérification manuelle et affichage des infos de maintenance restent à confirmer.
4 · Alertes / missions	Fonctions principales faites	Alertes et affectations opérationnelles ; conflits et statuts renforcés, alertes conducteur limitées à ses véhicules. Acceptation end-to-end encore ouverte.
5 · Utilisateurs	Partielle	L'admin consulte, crée et désactive. Modification des comptes absente ; libellés des rôles encore techniques.
6 · Dashboard	Partiel	KPI et répartition de flotte présents. Rapprochement des chiffres avec les données réelles de plusieurs entreprises à faire.
7 · Production	Partielle	Protections HTTP, contrôle JWT, transactions et index présents. Sauvegarde/restauration, CI et déploiement restent à préparer.
8 · Livraison	Non terminée	59 tests backend sur 11 suites et build frontend réussis au dernier relevé. Parcours E2E, accessibilité, responsive et validation explicite restent à faire.

Preuves récentes : **npm test** — 59 tests réussis sur 11 suites ; **npm run build** — build Vite réussi. Les tests automatisés ne remplacent pas les critères de validation manuelle ni l'acceptation explicite de chaque phase.

Hors périmètre MVP : application mobile native, Stripe/paiements, notifications email, PWA, géolocalisation active et exports avancés. Une liste ordonnée des vérifications et du travail restant figure en dernière page.

CarLog Pro

Dossier de projet — Micro SaaS de gestion de flotte automobile

Julie De Castro — Étudiante L3 Informatique, UPPA

Projet personnel — Objectif alternance septembre 2027

Ce document présente la démarche complète du projet CarLog Pro, depuis la compréhension du besoin métier jusqu'aux pistes d'évolution commerciale, conformément à une méthodologie de conception de produit SaaS.

1. Compréhension du métier

La gestion de flotte automobile (véhicules d'entreprise, utilitaires, engins) est aujourd'hui souvent gérée de façon artisanale : tableurs Excel, carnets papier, échanges par mail ou téléphone entre gestionnaires et conducteurs. Cette approche génère plusieurs problèmes récurrents chez les TPE/PME possédant un parc de quelques véhicules à quelques dizaines de véhicules :

- Maintenance oubliée ou faite trop tard (contrôle technique, vidange, pneus), entraînant pannes et immobilisations coûteuses.
- Absence de traçabilité : qui conduit quel véhicule, à quel moment, avec quel kilométrage.
- Alertes de sécurité ou d'échéances administratives (assurance, carte grise) non centralisées.
- Manque de visibilité globale sur l'état du parc pour le dirigeant ou le responsable de flotte.

Problématique retenue : comment permettre à une petite entreprise de piloter l'ensemble de son parc automobile (véhicules, conducteurs, maintenances, alertes) depuis un outil unique, simple, accessible en ligne, sans les coûts et la complexité des solutions de fleet management existantes destinées aux grands comptes (ex. Fleet Complete, Geotab), qui sont surdimensionnées et coûteuses pour une petite structure.

Positionnement micro SaaS : CarLog Pro cible volontairement un segment restreint (TPE/PME de 1 à 30 véhicules), avec un périmètre fonctionnel volontairement resserré plutôt qu'une couverture exhaustive du marché. Cela implique un développeur unique, une infrastructure légère et mutualisée (pas de microservices), et une priorité donnée à un cœur de produit solide plutôt qu'à la multiplication des fonctionnalités.

2. Identification des acteurs

Acteur	Rôle	Besoins principaux
Admin	Dirigeant / propriétaire du compte entreprise	Vue d'ensemble du parc, gestion des utilisateurs, facturation
Fleet manager	Responsable de flotte	Suivi des véhicules, planification des maintenances, affectations
Conducteur	Utilisateur du véhicule au quotidien	Consultation du véhicule affecté, signalement d'alertes
Mécanicien	Intervenant technique	Suivi et résolution des alertes de maintenance
Comptable	Gestion financière	Suivi des coûts, export de données

Cette segmentation par rôle (implémentée techniquement via le champ *role* du modèle User et le middleware *authorize*) garantit que chaque acteur n'accède qu'aux fonctionnalités pertinentes à sa fonction (RBAC — Role-Based Access Control).

3. Cahier des charges

3.1 Objectif du produit

Fournir une plateforme web (et à terme mobile) permettant à une entreprise de gérer son parc automobile de façon centralisée : véhicules, conducteurs, maintenances, alertes et statistiques.

3.2 Besoins fonctionnels

- Créer un compte entreprise et gérer plusieurs utilisateurs avec des rôles différents.
- Ajouter, modifier, consulter et archiver des véhicules.
- Affecter un véhicule à un conducteur sur une période donnée, avec suivi kilométrique.
- Créer et suivre des alertes (maintenance, carburant, sécurité, administratif) avec niveau d'urgence.
- Consulter un tableau de bord statistique (état du parc, affectations en cours).
- Isoler strictement les données de chaque entreprise (multi-tenant).

3.3 Besoins non fonctionnels

- Sécurité : authentification par token JWT, mots de passe hashés (bcrypt), contrôle d'accès par rôle.
- Performance : pagination et recherche indexées sur les listes de véhicules.
- Disponibilité : hébergement cloud avec objectif de disponibilité adapté à une TPE/PME (pas de SLA 24/7 strict en V1).
- Scalabilité limitée et assumée : architecture monolithique simple, suffisante pour le volume cible (quelques centaines d'entreprises, quelques dizaines de véhicules chacune).
- Conformité RGPD de base : données personnelles (conducteurs) limitées au nécessaire, possibilité de suppression de compte.

3.4 Contraintes

- Projet réalisé par une développeuse unique, en parallèle des études : contraintes de temps fortes.
- Budget d'infrastructure quasi nul en phase de développement (offres gratuites Render/Vercel/MongoDB Atlas).
- Stack imposée par l'apprentissage en cours : Node.js/Express, MongoDB, React.

4. Définition du MVP

Le MVP (Minimum Viable Product) est défini comme le plus petit ensemble de fonctionnalités permettant à une entreprise de remplacer son tableur Excel par CarLog Pro pour un usage quotidien réel.

Inclus dans le MVP	Exclu du MVP (V2+)
Inscription entreprise + authentification JWT	Paiement Stripe / abonnements payants
CRUD véhicules avec recherche et filtres	Application mobile native
CRUD alertes avec résolution	Notifications e-mail automatiques
Affectations véhicule / conducteur	Multi-tenant avancé (sous-organisations)
Statistiques de base du parc	Géolocalisation temps réel / géo-fencing actif
Gestion des rôles (RBAC)	Export comptable avancé

Ce périmètre correspond à l'état actuel de la Phase 1 (backend) du projet, quasiment complète, et à la Phase 2 (frontend React) à venir.

5. Conception de la base de données

Base de données NoSQL orientée documents (MongoDB Atlas + Mongoose), organisée autour de 5 collections principales, avec isolation multi-tenant par le champ *entreprise* présent sur les collections métier.

- **User** — utilisateurs, rattachés à une entreprise, avec rôle et mot de passe hashé.
- **Entreprise** — compte client (SaaS tenant), formule d'abonnement, statut.
- **Vehicule** — véhicules du parc, unicité de l'immatriculation par entreprise.
- **Alerte** — événements à traiter (maintenance, sécurité...), niveau d'urgence, statut.
- **Affectation** — historique et suivi des attributions véhicule / conducteur, kilométrage.

Des index composés (ex. $\{entreprise, statut\}$, $\{entreprise, immatriculation\}$) garantissent à la fois la cohérence multi-tenant et la performance des requêtes les plus fréquentes.

6. Développement du backend et de l'API

Statut : **terminé** pour le périmètre MVP.

- Node.js + Express, architecture en couches (routes / controllers / middlewares / models).
- Authentification JWT (token 7 jours) + hachage bcrypt (coût 12).
- Autorisation par rôle via middleware *authorize*.
- CRUD complet véhicules, alertes, affectations avec règles métier (ex. véhicule/conducteur déjà occupé, $kmFin \geq kmDebut$).
- Pagination, recherche et filtres sur les listes.
- Endpoint statistiques agrégées (agrégation MongoDB).
- Documentation API interactive via Swagger (/api-docs).
- Validation des entrées via express-validator.

7. Création de l'interface Web

Statut : **à venir** — Phase 2 de la roadmap.

- Stack : React.js (Vite), Axios pour la consommation de l'API REST, Context API pour la gestion de l'authentification.
- Pages prévues : inscription / connexion, dashboard, gestion véhicules, gestion alertes, gestion affectations.
- Bibliothèque de graphiques (Chart.js ou Recharts) pour la restitution visuelle des statistiques de flotte.
- Design responsive pour un usage bureau et tablette en priorité.

8. Création de l'application mobile

Choix assumé : pas d'application mobile native dans le périmètre du micro SaaS.

Pour un micro SaaS développé par une seule personne, maintenir deux front-ends (web + mobile natif ou React Native) représente une charge disproportionnée par rapport à la valeur ajoutée à ce stade. Le choix retenu est le suivant :

- L'interface web React sera conçue en responsive design, utilisable dans de bonnes conditions sur mobile via le navigateur (usage terrain des conducteurs pour consulter leur véhicule affecté ou signaler une alerte).
- Une déclinaison en PWA (Progressive Web App) est envisagée en V2 si le besoin d'un usage mobile plus poussé (mode hors-ligne, notifications push) se confirme auprès des premiers utilisateurs.
- Une application mobile native n'est pas exclue à terme, mais conditionnée à la validation du produit et à l'acquisition de premiers clients payants.

9. Authentification et gestion des rôles

Statut : **terminé**.

- Inscription : création simultanée d'une Entreprise et d'un User admin.
- Connexion : retour d'un token JWT valable 7 jours.
- Middleware *protect* : vérification du token sur les routes protégées, 401 si absent/invalid/expiré.
- Middleware *authorize(...roles)* : restriction d'accès par rôle, 403 si non autorisé.
- 5 rôles distincts : admin, fleet_manager, conducteur, mécanicien, comptable.

10. Tests fonctionnels et techniques

Statut : **partiel** — base posée, couverture à étendre.

- Fait : tests Jest + Supertest sur le module d'authentification (inscription réussie, gestion des doublons email), avec base de données de test dédiée (carlog_pro_test).
- À faire : tests sur les modules véhicules, alertes et affectations (création, permissions par rôle, règles métier comme $\text{kmFin} \geq \text{kmDebut}$), tests d'intégration multi-tenant (isolation stricte des données entre entreprises), tests end-to-end une fois le frontend disponible.

11. Conteneurisation

Statut : **non fait, prévu avant mise en production**.

- Objectif : conteneuriser le backend Node.js avec Docker (Dockerfile + docker-compose pour l'environnement de développement local, incluant une instance MongoDB locale pour ne pas dépendre d'Atlas en dev).
- Intérêt pour un micro SaaS solo : reproductibilité de l'environnement, déploiement simplifié sur Render (qui supporte le déploiement par Dockerfile), et facilité de montée en charge si migration future vers un autre hébergeur.

12. Déploiement

Statut : **planifié** — Phase 4 de la roadmap.

- Backend : Render (PaaS), variables d'environnement de production distinctes du développement.
- Frontend : Vercel, déploiement continu depuis GitHub.
- Base de données : MongoDB Atlas (cluster déjà en place).
- Nom de domaine personnalisé + HTTPS automatique.
- Mise en place d'un pipeline CI simple (GitHub Actions) pour lancer les tests avant chaque déploiement.

13. Sauvegarde

Statut : **non fait, à cadrer**. Pour un micro SaaS hébergeant des données d'entreprises clientes, une stratégie de sauvegarde minimale mais claire est nécessaire dès la mise en production :

- MongoDB Atlas : activation des sauvegardes automatiques continues (disponibles nativement sur Atlas, y compris sur certains tiers gratuits/bas coût selon le cluster choisi).
- Export périodique manuel (mongodump) en complément pendant la phase de lancement, stocké hors Atlas.
- Procédure de restauration testée au moins une fois avant l'arrivée des premiers clients réels.
- Politique de rétention simple (ex. 7 jours glissants) adaptée à la taille du projet, à faire évoluer si la base clients grandit.

14. Pistes d'évolution commerciale

14.1 Modèle économique

- Formules déjà modélisées en base : starter, premium, entreprise (champ *formuleAbonnement*).
- Statuts d'abonnement gérés : trial, active, past_due, canceled — base prête pour une intégration Stripe.
- Cible : TPE/PME avec parc de 1 à 30 véhicules, tarification mensuelle par véhicule ou par palier de flotte.

14.2 Étapes de mise sur le marché

- Intégration Stripe pour la facturation récurrente (abonnements + essai gratuit).
- Emails transactionnels (confirmation, relance d'essai, alertes critiques) via Resend/SendGrid.
- Acquisition initiale ciblée : artisans, PME de transport léger, sociétés de service avec véhicules utilitaires.
- Mise en avant du projet sur GitHub et LinkedIn comme vitrine technique pour la recherche d'alternance.

14.3 Évolutions produit envisageables

- PWA puis, si validé par le marché, application mobile native pour les conducteurs.
- Notifications automatiques (échéances contrôle technique, assurance).
- Export comptable et rapports PDF.

- Intégrations tierces (calendrier, cartes grises dématérialisées).

Synthèse

Volet	Statut
1. Compréhension du métier	Fait
2. Identification des acteurs	Fait
3. Cahier des charges	Fait
4. Définition du MVP	Fait
5. Conception de la base de données	Fait
6. Backend et API	Fait
7. Interface Web	Planifié
8. Application mobile	Exclu du périmètre (choix assumé)
9. Authentification et rôles	Fait
10. Tests fonctionnels et techniques	Partiel
11. Conteneurisation	Planifié
12. Déploiement	Planifié
13. Sauvegarde	Planifié
14. Pistes d'évolution commerciale	Fait (cadrage)

Travail restant — ordre recommandé

Les phases sont dépendantes : conserver cet ordre et faire valider chaque étape avant de considérer le MVP prêt à publier.

1. Fermer les phases 0 et 1 — environnement et backend

Vérifier le démarrage réel et l'accès Preview à l'API. Résoudre l'écart entre le design system clair décrit dans CLAUDE.md et l'interface sombre. Tester les parcours multi-entreprises et les cinq rôles ; obtenir l'acceptation explicite du socle.

2. Accepter les phases 2 à 4 — parcours opérationnels

Dans Preview, essayer inscription, mauvaises informations de connexion, actualisation de session, expiration du token et API indisponible. Tester le CRUD véhicule, PTAC vide/0, archivage, alertes, création/clôture/déplacement/suppression d'affectations et les conflits. Confirmer les données après actualisation. Décider si le véhicule doit afficher davantage d'informations de maintenance et si les alertes doivent filtrer par véhicule.

3. Compléter les phases 5 et 6 — utilisateurs et indicateurs

Décider si la modification et la réactivation d'un utilisateur sont requises ; présenter des noms de rôles lisibles. Comparer les KPI aux listes après création, clôture et archivage, avec plusieurs entreprises et rôles.

4. Préparer la phase 7 — production

Confirmer que les secrets restent côté backend ; nettoyer les variables backend présentes dans frontend/.env. Définir une sauvegarde, restaurer une sauvegarde de test, ajouter/valider une CI et documenter le déploiement et l'arrêt. Le dossier initial envisage Docker, mais aucun Dockerfile ni docker-compose n'est actuellement présent.

5. Exécuter la phase 8 — livraison

Depuis un environnement propre, exécuter tests et build, puis rejouer tous les parcours avec deux entreprises et les cinq rôles. Vérifier erreurs réseau, desktop/tablette/mobile, labels, clavier et contrastes. Mettre à jour la documentation et obtenir une validation explicite avant toute publication.

Documents et cohérence

Le README a été réécrit pour refléter React/Vite et l'API actuelle. Le dossier projet d'origine est conservé ; cette copie place la présente mise à jour en première page et en dernière page afin que les anciens statuts restent consultables sans ambiguïté. La checklist détaillée est dans docs/validation-et-prochaines-etapes.md.